FunkSpace — Agent Workflow Playbook (Cursor + Codex)

Version: 2025-11-04 · **Scope:** Home page animations is the running example, but this applies to any large feature.

Why this exists

A tiny playbook the FunkSpace Assistant can follow to plan and execute work **inside Cursor** (right sidebar Agent Chat) and **with Codex** (left sidebar IDE extension). It encodes guardrails, prompts, and reusable commands so agents stop guessing and start shipping.

Workflow overview (TL;DR)

1) **Plan in ChatGPT** (this FunkSpace Assistant): produce a one-pager spec in `docs/features/<feature>.md` and a checklist. 2) **Bootstrap in Cursor Agent Chat (right):** use **Plan Mode** to generate an implementation plan; run with foreground approvals. Toggle **Cloud Agent** only for long test/build loops. 3) **Focused passes with Codex (left):** use **Agent** mode for edits/runs; escalate to **Agent (Full Access)** only when strictly needed; optionally offload to cloud tasks. 4) **Automate ritual steps with `/commands`:** store repeatable flows under `.cursor/commands` for lint/tests/e2e, scaffolds, and PR routines. 5) **Ship behind a flag** and validate with perf + a11y + tests.

Assistant kickoff template (paste into ChatGPT before switching to Cursor)

```
Goal
- Feature: <name>
- User impact & acceptance criteria: <bullets>
- KPIs/guardrails: LCP/INP deltas, CLS ≤ 0.1, zero axe violations.

Plan shape
- Components to touch: <paths>
- Tokens/variables to add: <motion/spacing/color>
- Tests: unit on utils, Playwright smoke on key flows, axe checks.
- Rollout: env flag NEXT_PUBLIC_<FEATURE>_ENABLED, 10% ramp.

Artifacts to create
- docs/features/<feature>.md
```

```
- .cursor/commands/<feature>-setup.md
- app/(home)/components/<files>
```

---

1) Bootstrap with Cursor Agent Chat (right)

Modes and guardrails

- Start in **Foreground Agent** so every terminal command needs approval.
- Switch to **Cloud Agent** only for long iterative loops (tests/builds) where auto-run helps.
- Use **Plan Mode** to draft and review an implementation plan before edits.

Kickoff prompt (Cursor Agent)

Paste in Agent Chat:

```
Scan docs/features/<feature>.md and propose a step-by-step plan in Plan Mode.
Then:
1) Create branch feat/<feature>.
2) Scaffold files: app/(home)/components/HeroMotion.tsx, FeatureCardMotion.tsx,
utils/motion.ts.
3) Add motion tokens and Tailwind theme entries (durations/easings; respect
prefers-reduced-motion).
4) Write Vitest for motion utils and Playwright smoke for hero + card
animations.
5) Wire axe checks; fail on violations.
Ask before running any command. Summarize diffs before applying.
```

Approvals policy

- Allow only: `pnpm`, `eslint`, `vitest`, `playwright`, `lighthouse-ci`.
- Deny network unless explicitly required by the task.

Validation loop

- Lint + types: `pnpm lint && pnpm typecheck`
- Unit: `pnpm test -u`
- E2E smoke: `pnpm e2e:smoke`
- Perf: Lighthouse CI on the home route (with motion on/off)

Notes for large edits

- Use **Plan Mode** for multi-file refactors.
- For flaky tests, temporarily flip to **Cloud Agent** to auto-iterate, then return to approvals-on.

---

2) Bring in Codex (left) as a focused specialist

When to use

- Micro-optimizations, targeted refactors, a11y/perf polish, or a second opinion on a stuck diff.

Modes

- **Chat** for ideation/exploration.
- **Agent** for read/edit/run inside the workspace automatically.
- **Agent (Full Access)** only when you truly need network access or broader FS. De-escalate after the task.
- Adjust **Reasoning Effort**: default to **Medium**; raise to **High** only for complex changes.

Quick prompts (Codex)

- "Review `HeroMotion.tsx` for layout thrash; replace transforms that trigger reflow; propose minimal variants respecting reduced-motion."
- "Optimize `motion.ts` durations/easing for smoothness on low-end devices; keep 60–120 fps; update tests if needed."
- "Open a cloud task to profile first paint and produce a perf report; return with proposed diffs."

Safety toggles

- Prefer local workspace runs; only enable network or cloud when necessary.
- Always inspect diffs and logs before applying.

---

3) Encode repeatable flows as /commands

Place markdown commands in `.cursor/commands/`. Each command is a short, deterministic playbook the agent can invoke by `/name`.

Example A — `animations-setup.md`

```
# Setup: Home Animations
- pnpm add framer-motion
- Create utils/motion.ts with exported tokens: durations, easings, spring
presets.
- Tailwind theme: motion durations/easings as CSS vars.
- Scaffold components: HeroMotion.tsx, FeatureCardMotion.tsx
- Update docs/motion.md with tokens and a11y rules.
- Run: pnpm lint && pnpm typecheck
```

Example B — `qa-smoke.md`

```
# QA Smoke
- pnpm lint && pnpm typecheck
- pnpm test -u
- pnpm e2e:smoke
- Run axe checks; fail on violations
- Save Playwright traces/videos to artifacts/
```

Example C — `pr-checklist.md`

```
# PR Checklist
- Summarize diffs and breaking changes
- Verify env flag defaults OFF
- Lighthouse CI: attach JSON
- Axe: zero serious violations
- Include "How to test" steps
```

> Tip: keep commands tiny and composable; chain them in chat rather than writing mega-scripts.

A11y + Perf rules for motion (baseline)

- Respect `prefers-reduced-motion`; provide static fallbacks.
- Avoid layout-thrashing properties in animations; favor `transform` + `opacity`.
- Cap total intro motion ≤ 400ms for primary interactions; keep Easing consistent via tokens.
- No CLS introduced by animating layout.

Release discipline

1) Feature flag: `NEXT_PUBLIC_ANIMATIONS_ENABLED`. 2) 10% rollout; watch LCP/INP/CLS and error budget. 3) Remove flag after metrics hold for 24–48h.

Safety checklist (agents)

- Keep approvals ON by default; temporarily enable auto-run only for tight loops.
- Treat network access as privileged; disable by default.
- Review every diff; never commit without green lint, types, unit, and smoke tests.

Minimal file scaffold

```
/docs/features/home-animations.md     ← spec one-pager
/.cursor/commands/animations-setup.md ← scaffold
/.cursor/commands/qa-smoke.md         ← tests
/.cursor/commands/pr-checklist.md     ← governance
/app/(home)/components/HeroMotion.tsx
/app/(home)/components/FeatureCardMotion.tsx
/utils/motion.ts
```

Done / Next / Blockers

- **Done:** Playbook + commands skeleton ready.
- **Next:** Fill the feature spec and run the Cursor plan; follow the commands.
- **Blockers:** None, unless CI or permissions are misconfigured.